

Taller — Predicción de accidentalidad

Clasificación binaria · Clases desbalanceadas · Datos en SQLite

Tema	¿Habrà al menos un accidente en el barrio B a la hora h ?
Profesor	_____
Modalidad	_____
Fecha de entrega	_____
Datos	accidentes_taller.sqlite (archivo adjunto a este taller)
Notebook guía	taller_accidentalidad_alumnos.ipynb — complete este notebook; ya trae la carga de datos, funciones auxiliares y pistas
Entregable	Un único notebook .ipynb con el código y las respuestas en celdas de texto
Tiempo estimado	Entre 6 y 8 horas de trabajo

Cómo leer este taller

Cada actividad está marcada como **[ESENCIAL]** u **[OPCIONAL · BONO]**. Las actividades esenciales suman los 100 puntos de la nota. Las opcionales dan hasta 10 puntos extra (sin exceder la nota máxima) y solo deben abordarse después de terminar las esenciales.

1. Contexto

Una secretaría de movilidad quiere anticipar **dónde y cuándo** es más probable que ocurra un accidente de tránsito para ubicar mejor a sus agentes. La dificultad es que en la gran mayoría de combinaciones (barrio, hora) **no** ocurre ningún accidente: solo una pequeña fracción es positiva. Es un problema de **clases desbalanceadas**, y ahí está el aprendizaje central del taller: entender por qué la *accuracy* engaña y cómo elegir métricas, pesos de clase y umbral de decisión.

2. Datos

La base **accidentes_taller.sqlite** tiene dos tablas que cubren 20 barrios entre el 1 de enero y el 30 de abril de 2025. *Los datos son simulados con fines académicos*: los nombres de los barrios son reales, pero los registros no corresponden a accidentes reales.

Tabla accidentes — una fila por accidente reportado (≈ 4.300 filas)

Columna	Tipo	Descripción
ID	INTEGER	Identificador del registro
FECHA_HORA	TEXT	Fecha y hora exacta del accidente, al minuto (ej. 2025-02-12 12:15:00)
BARRIO	TEXT	Barrio donde ocurrió
CLASE	TEXT	Tipo de accidente (Choque, Atropello, ...)
GRAVEDAD	TEXT	Solo daños, Con heridos, Con muertos

Tabla clima — una fila por cada (barrio, hora) del período (57.600 filas)

Columna	Tipo	Descripción
TW	TEXT	Marca de tiempo truncada a la hora (ej. 2025-02-12 12:00:00)
BARRIO, COMUNA	TEXT	Barrio y la comuna a la que pertenece
temperatura	REAL	Temperatura en °C
humedad	REAL	Humedad relativa en %
precipitacion	REAL	Lluvia en mm durante esa hora (0 = no llovió)
visibilidad	REAL	Visibilidad en km
viento	REAL	Velocidad del viento en m/s

Idea clave. La tabla `clima` contiene **todas** las combinaciones (barrio, hora); la tabla `accidentes` solo contiene los casos donde **sí** hubo accidente. La variable objetivo es **1** si en esa pareja (barrio, hora) ocurrió al menos un accidente, y **0** en caso contrario.

3. Actividades

Responda cada pregunta en una celda Markdown del notebook, justo después del código que la respalda. Las respuestas escritas deben ser breves: entre 2 y 5 líneas.

Parte 1 · Carga y calidad de los datos (15 pts) [ESENCIAL]

- Conéctese a la base con `sqlite3` y cargue ambas tablas con consultas SQL (ver plantilla en la sección 4). No use archivos CSV intermedios.
- Los datos tienen **tres tipos de problemas** que usted debe encontrar y corregir. Para cada uno, reporte cuántos registros afecta antes de corregirlo y cómo lo resolvió:
 - **Nombres de barrio inconsistentes** en accidentes (compare con los de `clima`).
 - **Registros duplicados** en accidentes.
 - **Valores faltantes e imposibles** en las variables de clima. Decida cómo imputarlos y justifique.

Parte 2 · Construcción de la variable objetivo (15 pts) [ESENCIAL]

- Trunque `FECHA_HORA` a la hora para que coincida con `TW` (pista: `.dt.floor('h')`). Si en una misma hora y barrio hubo varios accidentes, cuéntelos como un único positivo.
- Una las tablas por (`BARRIO`, `TW`) para obtener un DataFrame con una fila por (barrio, hora) y la columna `target` (0/1).
- **Pregunta:** ¿qué tipo de *join* usó y desde qué tabla? ¿Qué pasaría con los casos negativos si usara un *inner join*?
- **Pregunta:** ¿qué porcentaje de filas es positivo? ¿Qué implica eso para el modelado?

Parte 3 · Análisis exploratorio (15 pts) [ESENCIAL]

Haga exactamente **tres gráficos**, cada uno con título y ejes rotulados, y escriba una conclusión de 1 o 2 líneas debajo de cada uno:

- Proporción de horas con accidente según la **hora del día** (0 a 23).
- Proporción de horas con accidente por **barrio**, ordenada de mayor a menor.
- Proporción de horas con accidente **con lluvia** vs. **sin lluvia**.

Parte 4 · Variables (features) (10 pts) [ESENCIAL]

- Cree como mínimo: hora del día, día de la semana, indicador de fin de semana y un indicador llueve (precipitación mayor que 0).
- Codifique BARRIO con *one-hot encoding*.
- Incluya las variables climáticas que considere útiles según su análisis exploratorio.
- [OPCIONAL · BONO] Codifique la hora y el día de la semana con seno y coseno, y explique por qué puede ayudar.

Parte 5 · Modelado con clases desbalanceadas (30 pts) [ESENCIAL]

Partición temporal: entrene con enero–marzo y evalúe con abril. No use una partición aleatoria: en un problema con tiempo, eso usa el futuro para predecir el pasado.

- **a) Baseline.** Calcule la *accuracy* en test de un modelo que siempre predice 0. **Pregunta:** ¿por qué este número es engañoso?
- **b) Regresión logística** dos veces: sin balanceo y con `class_weight='balanced'`. Recuerde escalar las variables numéricas.
- **c) Un modelo de ensamble** a su elección (Random Forest, Gradient Boosting, XGBoost o LightGBM) con alguna estrategia de balanceo (`class_weight` o `scale_pos_weight`). Ajuste sus hiperparámetros con GridSearchCV usando TimeSeriesSplit sobre los datos de entrenamiento y `scoring='average_precision'`. Límite: máximo 2 hiperparámetros con 3 valores cada uno.
- **d) Tabla comparativa** de los modelos en el conjunto de test con: PR-AUC, ROC-AUC, y precisión, recall y F1 de la clase positiva con umbral 0,5.
- **Pregunta:** ¿qué cambió al usar `class_weight='balanced'`? ¿Por qué la PR-AUC es más informativa que la ROC-AUC en este problema?
- [OPCIONAL · BONO] Compare también con **submuestreo aleatorio** de la clase mayoritaria (solo en entrenamiento) o con un tercer tipo de modelo.

Referencia para interpretar resultados. Un modelo que asigna probabilidades al azar obtiene una PR-AUC aproximadamente igual a la proporción de positivos. Su modelo es útil en la medida en que supere claramente ese valor; no espere valores cercanos a 1, porque los accidentes tienen mucho de aleatorio.

Parte 6 · Umbral de decisión y caso de uso (15 pts) [ESENCIAL]

- Elija su mejor modelo y justifique la elección en 2 o 3 líneas.
- Suponga que para la secretaría es mucho más costoso **no detectar** un accidente que enviar un agente a un lugar donde no pasa nada. Encuentre el umbral que logra un **recall de al menos 0,60** en test (pista: `precision_recall_curve`) y reporte la precisión que obtiene con él.
- Muestre la **matriz de confusión** con ese umbral e interprete en palabras los falsos positivos y los falsos negativos.
- **Caso de uso:** para un día de abril elegido por usted, liste las 10 combinaciones (barrio, hora) con mayor probabilidad predicha. ¿Cuántas de ellas tuvieron realmente un accidente?
- **Limitaciones:** en un párrafo de 3 a 5 líneas, mencione al menos dos limitaciones del modelo y una variable externa que, si estuviera disponible, podría mejorarlo.

4. Entregable

Un único notebook **.ipynb** que cumpla con lo siguiente:

- Se ejecuta completo, en orden y sin errores (*Kernel* → *Restart & Run All*).
- Usa una ruta relativa a la base de datos (el archivo `.sqlite` en la misma carpeta del notebook).
- Fija `random_state=42` en todo lo que tenga aleatoriedad.
- Tiene títulos por parte (Parte 1, Parte 2, ...) y las respuestas en celdas Markdown.
- El código está comentado en español.

Plantilla de conexión sugerida:

```
import sqlite3
import pandas as pd

con = sqlite3.connect('accidentes_taller.sqlite')
accidentes = pd.read_sql('SELECT * FROM accidentes', con, parse_dates=['FECHA_HORA'])
clima = pd.read_sql('SELECT * FROM clima', con, parse_dates=['TW'])
con.close()
```

5. Rúbrica de evaluación

Cada ítem se califica como cumplido o no cumplido, con los puntos indicados.

Parte	Pts	Qué debe estar en el notebook
1. Calidad de datos	15	Carga con SQL (3) · Barrios normalizados, con conteo antes/después (4) · Duplicados detectados y eliminados (4) · Faltantes e imposibles tratados y justificados (4)
2. Variable objetivo	15	Truncado a la hora correcto (4) · Join correcto con una fila por (barrio, hora) (6) · Respuesta sobre el join (2) · % de positivos reportado e interpretado (3)
3. Exploratorio	15	Tres gráficos correctos y legibles (3 × 3) · Conclusión escrita para cada uno (3 × 2)
4. Variables	10	Variables temporales y de lluvia (4) · One-hot de barrio (3) · Selección de clima justificada (3)
5. Modelado	30	Partición temporal correcta (5) · Baseline y respuesta (4) · Logística con y sin balanceo (6) · Ensamble con GridSearchCV + TimeSeriesSplit (7) · Tabla comparativa completa (4) · Respuestas (4)
6. Umbral y caso de uso	15	Modelo elegido con justificación (2) · Umbral con recall $\geq 0,60$ y precisión reportada (4) · Matriz de confusión interpretada (3) · Top-10 con aciertos (3) · Limitaciones (3)
Total	100	Si el notebook no se ejecuta completo, se descuentan 10 puntos.
Bono (opcional)	+10	Codificación seno/coseno explicada (5) · Submuestreo o tercer modelo comparado (5)